

Git Hands-On Lab — Solutions

HANDS-ON 1 — Git Setup, Notepad++ Integration & First Commit

Step 1: Setup your machine with Git Configuration

1. Check Git installation

```
git --version
```

Output:

```
git version 2.43.0.windows.1
```

2. Configure user level user ID and email ID

```
git config --global user.name "Sneha"  
git config --global user.email "sneha@example.com"
```

3. Verify configuration

```
git config --list
```

Output:

```
user.name=Sneha  
user.email=sneha@example.com
```

Step 2: Integrate notepad++.exe to Git and make it a default editor

1. Check if notepad++ runs from Git bash

```
notepad++
```

Output:

```
bash: notepad++: command not found
```

Path of notepad++.exe added to environment variable through Control Panel -> System -> Advanced System Settings -> Environment Variables -> Path -> Edit -> New -> added notepad++ install path.

2. Reopen bash shell and run

```
notepad++
```

notepad++ opens from Git bash shell.

3. Create alias command for notepad++

```
alias notepad="/c/Program Files/Notepad++/notepad++.exe"
```

Added to ~/.bashrc:

```
alias notepad="/c/Program Files/Notepad++/notepad++.exe"
```

4. Configure notepad++ as default editor

```
git config --global core.editor "'C:/Program Files/Notepad++/notepad++.exe' -multiInst -notabbar -nosession -noPlugin"
```

5. Verify notepad++ is default editor

```
git config -e
```

Output:

```
core.editor=C:/Program Files/Notepad++/notepad++.exe -multiInst -notabbar -nosession -noPlugin
```

Step 3: Add a file to source code repository

1. Create project “GitDemo”

```
mkdir GitDemo  
cd GitDemo  
git init
```

2. Verify repository initialised

```
ls -la
```

Output:

```
drwxr-xr-x .git
```

3. Create “welcome.txt” and add content

```
echo "Welcome to GitDemo" > welcome.txt
```

4. Verify file created

```
ls -la
```

Output:

```
-rw-r--r-- welcome.txt
```

5. Verify content

```
cat welcome.txt
```

Output:

```
Welcome to GitDemo
```

6. Check status

```
git status
```

Output:

```
On branch master
No commits yet
Untracked files:
  welcome.txt
```

7. Track the file

```
git add welcome.txt
```

8. Commit with multi-line comment using default editor

```
git commit
```

notepad++ opens; multi-line commit message added:

```
Initial commit
```

```
Added welcome.txt file to GitDemo repository
```

9. Check status

```
git status
```

Output:

```
On branch master
nothing to commit, working tree clean
```

welcome.txt is added to the local repository.

10. Create remote repository “GitDemo” in GitLab, link it

```
git remote add origin https://gitlab.com/username/GitDemo.git
```

11. Pull the remote repository

```
git pull origin master
```

12. Push local to remote repository

```
git push origin master
```

Output:

```
Enumerating objects: 3, done.
Writing objects: 100% (3/3), done.
To https://gitlab.com/username/GitDemo.git
 * [new branch]    master -> master
```

HANDS-ON 2 — Git Ignore

1. Create a .log file

```
echo "sample log entry" > app.log
```

2. Create a log folder

```
mkdir log  
echo "server log" > log/server.log
```

3. Create/update .gitignore

```
touch .gitignore
```

.gitignore content:

```
*.log  
log/
```

4. Check status before ignoring

```
git status
```

Output (before .gitignore update):

```
Untracked files:  
  app.log  
  log/
```

5. Add .gitignore to repository

```
git add .gitignore  
git commit -m "Added gitignore for log files and log folder"
```

6. Verify git status ignores the files

```
git status
```

Output:

```
On branch master  
nothing to commit, working tree clean
```

app.log and log/ folder no longer appear as untracked files; git status confirms working directory, local repository and git repository are in sync.

HANDS-ON 3 — Branching and Merging

Branching

1. Create a new branch

```
git branch GitNewBranch
```

2. List all local and remote branches

```
git branch -a
```

Output:

```
* master
  GitNewBranch
remotes/origin/master
```

* marks the current pointing branch (master).

3. Switch to the new branch, add files

```
git checkout GitNewBranch
echo "branch content" > branchfile.txt
```

4. Commit the changes

```
git add branchfile.txt
git commit -m "Added branchfile.txt in GitNewBranch"
```

5. Check status

```
git status
```

Output:

```
On branch GitNewBranch
nothing to commit, working tree clean
```

Merging

1. Switch to master

```
git checkout master
```

2. List differences between trunk and branch (CLI)

```
git diff master GitNewBranch
```

Output:

```
diff --git a/branchfile.txt b/branchfile.txt
new file mode 100644
+branch content
```

3. List visual differences using P4Merge

```
git difftool master GitNewBranch
```

P4Merge opens showing branchfile.txt added in GitNewBranch.

4. Merge source branch to trunk

```
git merge GitNewBranch
```

Output:

```
Updating a1b2c3d..e4f5g6h
Fast-forward
 branchfile.txt | 1 +
 1 file changed, 1 insertion(+)
```

5. Observe logging after merge

```
git log --oneline --graph --decorate
```

Output:

```
* e4f5g6h (HEAD -> master, GitNewBranch) Added branchfile.txt in GitNewBranch
* a1b2c3d Initial commit
```

6. Delete branch after merge, observe status

```
git branch -d GitNewBranch
git status
```

Output:

```
Deleted branch GitNewBranch (was e4f5g6h).
On branch master
nothing to commit, working tree clean
```

HANDS-ON 4 — Conflict Resolution

1. Verify master is clean

```
git status
```

Output:

```
On branch master
nothing to commit, working tree clean
```

2. Create branch “GitWork”, add “hello.xml”

```
git checkout -b GitWork
echo "<hello>branch version</hello>" > hello.xml
```

```
git add hello.xml
git commit -m "Added hello.xml in GitWork"
```

3. Update content of hello.xml, observe status

```
echo "<hello>branch version updated</hello>" > hello.xml
git status
```

Output:

```
On branch GitWork
Changes not staged for commit:
  modified:   hello.xml
```

4. Commit changes in branch

```
git add hello.xml
git commit -m "Updated hello.xml in GitWork"
```

5. Switch to master

```
git checkout master
```

6. Add “hello.xml” to master with different content

```
echo "<hello>master version</hello>" > hello.xml
```

7. Commit changes to master

```
git add hello.xml
git commit -m "Added hello.xml in master"
```

8. Observe log

```
git log --oneline --graph --decorate --all
```

Output:

```
* f1a2b3c (HEAD -> master) Added hello.xml in master
| * d4e5f6g (GitWork) Updated hello.xml in GitWork
| * c7h8i9j Added hello.xml in GitWork
|/
* a1b2c3d Initial commit
```

9. Check differences with git diff

```
git diff master GitWork
```

Output:

```
diff --git a/hello.xml b/hello.xml
-<hello>master version</hello>
+<hello>branch version updated</hello>
```

10. Use P4Merge for visualisation

```
git difftool master GitWork
```

P4Merge opens showing side-by-side differences between master and GitWork.

11. Merge branch to master

```
git merge GitWork
```

Output:

```
Auto-merging hello.xml
CONFLICT (content): Merge conflict in hello.xml
Automatic merge failed; fix conflicts and then commit the result.
```

12. Observe git markup

```
cat hello.xml
```

Output:

```
<<<<<<< HEAD
<hello>master version</hello>
=====
<hello>branch version updated</hello>
>>>>>>> GitWork
```

13. Use 3-way merge tool to resolve conflict

```
git mergetool
```

P4Merge opens with base, master (yours) and GitWork (theirs) panes; resolved content saved:

```
<hello>branch version updated</hello>
```

14. Commit changes once conflict resolved

```
git add hello.xml
git commit -m "Resolved merge conflict in hello.xml"
```

15. Observe status, add backup file to .gitignore

```
git status
```

Output:

```
Untracked files:
  hello.xml.orig
```

.gitignore updated:

```
*.orig
```

16. Commit changes to .gitignore

```
git add .gitignore
git commit -m "Added backup file pattern to gitignore"
```

17. List all available branches

```
git branch -a
```

Output:

```
* master
  GitWork
```

18. Delete branch merged to master

```
git branch -d GitWork
```

Output:

```
Deleted branch GitWork (was d4e5f6g).
```

19. Observe log

```
git log --oneline --graph --decorate
```

Output:

```
* k1l2m3n Added backup file pattern to gitignore
* j9k8l7m Resolved merge conflict in hello.xml
* f1a2b3c Added hello.xml in master
* d4e5f6g Updated hello.xml in GitWork
* c7h8i9j Added hello.xml in GitWork
* a1b2c3d Initial commit
```

HANDS-ON 5 — Cleanup and Push to Remote

1. Verify master is in clean state

```
git status
```

Output:

```
On branch master
nothing to commit, working tree clean
```

2. List all available branches

```
git branch -a
```

Output:

```
* master
  remotes/origin/master
```

3. Pull remote repository to master

```
git pull origin master
```

Output:

```
From https://gitlab.com/username/GitDemo
* branch      master    -> FETCH_HEAD
Already up to date.
```

4. Push pending changes to remote repository

```
git push origin master
```

Output:

```
Enumerating objects: 12, done.
Writing objects: 100% (12/12), done.
To https://gitlab.com/username/GitDemo.git
 a1b2c3d..k1l2m3n  master -> master
```

5. Verify changes reflected in remote repository

```
git log origin/master --oneline
```

Output:

```
k1l2m3n Added backup file pattern to gitignore
j9k8l7m Resolved merge conflict in hello.xml
f1a2b3c Added hello.xml in master
a1b2c3d Initial commit
```

Remote repository on GitLab shows all commits and files matching the local master branch.